

作业 HW2 实验报告

姓名：汪嘉晨 学号：2353900 日期：2024 年 10 月 19 日

1. 涉及数据结构和相关背景

一、栈 (Stack)

- 定义：**栈是一种后进先出 (LIFO, Last In First Out) 的数据结构。它只允许在有序线性数据集合的一端 (称为栈顶, top) 进行插入数据 (PUSH) 和删除数据 (POP) 的运算。
- 特点：**
 - 数据存储在内存的连续空间中。
 - 数据的插入和删除都发生在栈顶，栈顶是可变的。
 - 栈中元素个数为零时称为空栈。
- 主要操作：**
 - PUSH (入栈)：**将数据插入栈顶。
 - POP (出栈)：**从栈顶删除数据。
 - PEEK (查看栈顶)：**查看栈顶数据但不删除。
 - ISEMPTY (判断栈空)：**判断栈是否为空。
- 应用场景：**栈主要用于实现程序的调用和返回，以及实现表达式的求值等功能。例如，在函数调用时，系统会使用栈来存储断点，以便在函数返回时能够恢复到正确的执行位置。
- 实现方式：**栈可以使用数组或链表来实现。使用数组实现的栈称为顺序栈，使用链表实现的栈称为链栈。

二、队列 (Queue)

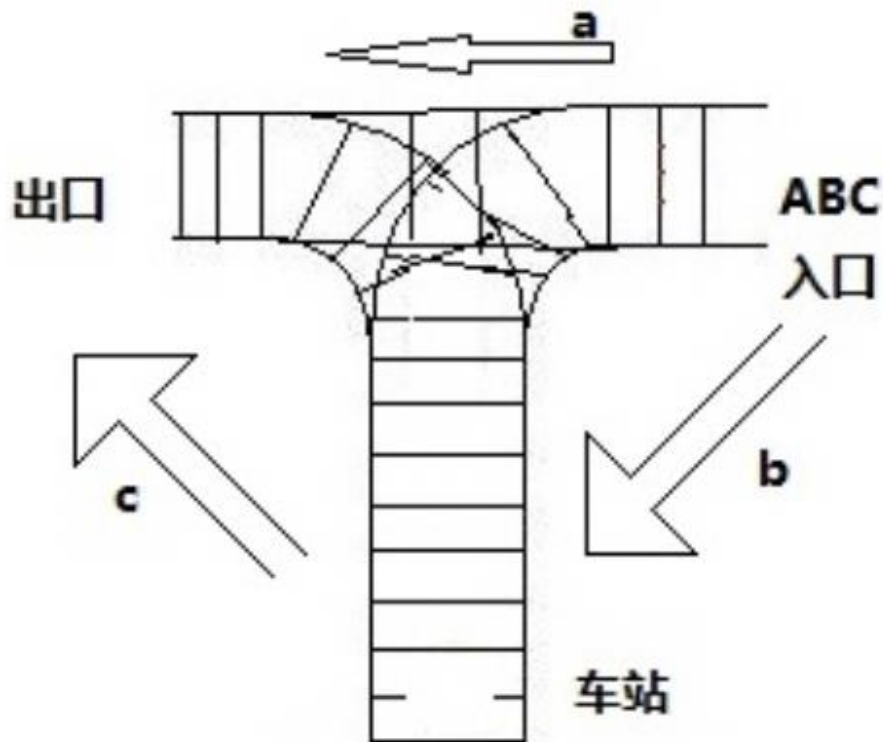
- 定义：**队列是一种先进先出 (FIFO, First In First Out) 的数据结构。它只允许在队列的一端 (称为队尾, rear) 进行插入数据 (ENQUEUE) 的操作，在另一端 (称为队头, front) 进行删除数据 (DEQUEUE) 的操作。
- 特点：**
 - 数据存储在内存的连续空间中。
 - 数据的插入发生在队列尾部，数据的删除发生在队列头部。
 - 队列头部和队列尾部是可变的。
- 主要操作：**
 - ENQUEUE (入队)：**将数据插入队列尾部。
 - DEQUEUE (出队)：**从队列头部删除数据。
 - PEEK (查看队头)：**查看队列头部数据但不删除。
 - ISEMPTY (判断队列空)：**判断队列是否为空。
- 应用场景：**队列主要用于实现进程调度、缓冲区管理等功能。例如，在计算机系统的任务调度中，队列被用来存储等待执行的任务，系统按照先进先出的原则依次执行这些任务。
- 实现方式：**队列同样可以使用数组或链表来实现。使用数组实现的队列称为顺序队列，使用链表实现的队列称为链式队列。其中，链式队列在实现上更加灵活，因为插入和删除操作只需要调整指针即可，而不需要移动大量的数据。

2. 实验内容

2.1 列车进站

2.1.1 问题描述

每一时刻，列车可以从入口进车站或直接从入口进入出口，再或者从车站进入出口。即每一时刻可以有一辆车沿着箭头 a 或 b 或 c 的方向行驶。现在有一些车在入口处等待，给出该序列，然后给你多组出站序列，请你判断是否能够通过上述的方式从出口出来。



2.1.2 基本要求

输入：

第 1 行，一个串，进站序列。

后面多行，每行一个串，表示出栈序列

当输入=EOF 时结束

输出：

多行，若给定的出栈序列可以得到，输出 yes,否则输出 no。

2.1.3 数据结构设计

//判断给定的出栈序列是否可以通过入栈序列得到

```
bool isValidSequence(const string& in, const string& out) {
```

```
    stack<char> station; //车站的栈
```

```
    int j = 0; //出栈序列的指针
```

```
    //遍历入栈序列
```

```
    for (char ch : in)
```

```

{
    station.push(ch); //将当前字符入栈，进入栈顶

    //检查栈顶元素是否与出栈序列的当前元素匹配
    while (!station.empty() && station.top() == out[j])
    {
        station.pop(); // 如果匹配则出栈
        j++; //移动出栈序列的指针
    }
}

// 如果栈为空，说明出栈序列是可行的
return station.empty();
}

```

2.1.4 功能说明（函数、类）

```

bool isValidSequence(const string& in, const string& out) {}

int main()
{
    string in;
    cin >> in; //读取入栈序列
    string out;
    //读取多个出栈序列，直到EOF
    while (cin >> out)
    {
        //判断当前出栈序列是否可行，并输出结果
        if (out == "EOF")
        {
            break;
        }
        else if (isValidSequence(in, out))
        {
            cout << "yes" << endl;
        }
        else
        {
            cout << "no" << endl;
        }
    }

    return 0;
}

```

2.1.5 调试分析（遇到的问题 and 解决方法）

```
C:\Users\wangj\Desktop\数据 × + v
abc
abc
yes
acb
yes
abbcc
no
cba
yes
```

2.1.6 总结和体会

难点分析：

1. 栈操作的理解：需要理解栈的后进先出（LIFO）特性，并正确应用到问题中。
2. 模拟过程：需要准确模拟列车进站和出站的过程，确保每一步操作都符合题目要求。
3. 边界条件：需要处理各种边界条件，如空入栈序列、空出栈序列、入栈序列和出栈序列长度不匹配等。

通过这道题目，可以巩固对栈的基本操作和模拟算法的理解，还能提高处理边界条件和优化算法效率的能力。

2.2 布尔表达式

2.2.1 问题描述

计算如下布尔表达式 $(V \mid V) \& F \& (F \mid V)$ 其中 V 表示 True，F 表示 False，| 表示 or，& 表示 and，! 表示 not（运算符优先级 not > and > or）

2.2.2 基本要求

输入

有若干 ($A \leq 20$) 个表达式，其中每一行为一个表达式。表达式有 ($N \leq 100$) 个符号，符号间可以用任意空格分开，或者没有空格，所以表达式的总长度，即字符的个数，是未知的。

对于 20% 的数据，有 $A \leq 5$ ， $N \leq 20$ ，且表达式中包含 V、F、&、|

对于 40% 的数据，有 $A \leq 10$ ， $N \leq 50$ ，且表达式中包含 V、F、&、|、!

对于 100% 的数据，有 $A \leq 20$ ， $N \leq 100$ ，且表达式中包含 V、F、&、|、!、(、)

所有测试数据中都可能穿插空格

下载并运行 96.py 生成随机测试数据

输出：

对测试用例中的每个表达式输出“Expression”，后面跟着序列号和“:”，然后是相应的测试表达式

的结果（V 或 F），每个表达式结果占一行（注意冒号后面有空格）。

2.2.3 数据结构设计

//T, F所成的栈

```
stack<bool> boolStack;
```

//操作符所成的栈

```
stack<char> opStack;
```

2.2.4 功能说明（函数、类）

判断先后顺序：

```
void processOperator() {
    char op = opStack.top();
    opStack.pop();
    if (op == '|') {
        bool b1 = boolStack.top();
        boolStack.pop();
        bool b2 = boolStack.top();
        boolStack.pop();
        boolStack.push(b1 || b2);
    }
    else if (op == '&') {
        bool b1 = boolStack.top();
        boolStack.pop();
        bool b2 = boolStack.top();
        boolStack.pop();
        boolStack.push(b1 && b2);
    }
    else if (op == '!') {
        bool b = boolStack.top();
        boolStack.pop();
        boolStack.push(!b);
    }
}

//读取输入的字符串
void procession(string str)
{
    for (int i = 0; i < str.size(); i++)
    {
        if (str[i] ==
            boolStack.push(true);
        }
        else if (str[i] == 'F') //读到F，入栈boolStack
        {
```

```

        boolStack.push(false);
    }
    else if (str[i] == '(') //读到(, 入栈opStack
    {
        opStack.push(str[i]); 'V' //读到V, 入栈boolStack
    }
    else if (str[i] == ')') //读到), 计算括号内的表达式
    {
        while (opStack.top() != '(')
        {
            processOperator();
        }
        opStack.pop(); //弹出左括号
    }
    else if (str[i] == '|') //读到|, 入栈opStack
    {
        while (!opStack.empty() && opStack.top() != '(' && opStack.top() !=
'|')
        {
            processOperator();
        }
        opStack.push(str[i]);
    }
    else if (str[i] == '&') //读到&, 判断栈顶是否为|, 如果是则先计算|, 再入栈&
    {
        while (!opStack.empty() && opStack.top() != '(' && opStack.top() !=
'|' && opStack.top() != '&')
        {
            processOperator();
        }
        opStack.push(str[i]);
    }
    else if (str[i] == '!') //读到!, 立即处理
    {
        opStack.push(str[i]);
    }
    else if (str[i] == ' ') //读到空格, 跳过
    {
        continue;
    }
}
while (!opStack.empty()) //处理剩余的操作符
{

```

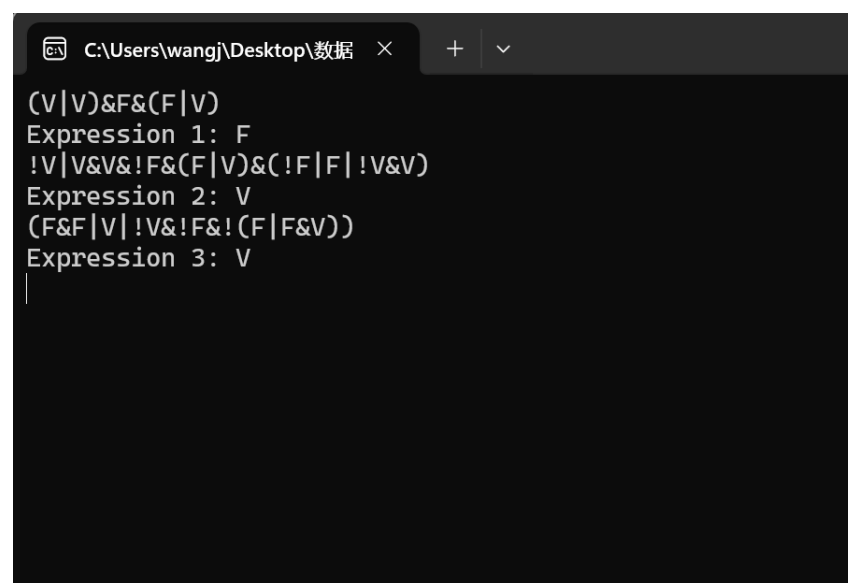
```

        processOperator();
    }
}

//主函数，循环读取字符串，并输出结果
int main()
{
    string str;
    int index = 1;
    while (getline(cin, str))
    {
        procession(str);
        if (boolStack.top())
        {
            cout << "Expression " << index << ": V" << endl;
        }
        else
        {
            cout << "Expression " << index << ": F" << endl;
        }
        //清空栈以处理下一个表达式
        while (!boolStack.empty()) boolStack.pop();
        while (!opStack.empty()) opStack.pop();
        index++;
    }
    return 0;
}

```

2.2.5 调试分析（遇到的问题解决方法）



```

C:\Users\wangj\Desktop\数据
(V|V)&F&(F|V)
Expression 1: F
!V|V&V&!F&(F|V)&(!F|F|!V&V)
Expression 2: V
(F&F|V|!V&!F&!(F|F&V))
Expression 3: V
|

```

2.2.6 总结和体会

表达式解析：

通过遍历输入字符串，将布尔值和操作符分别压入两个栈中。

使用栈来处理操作符的优先级和括号的嵌套。

操作符优先级：

! 的优先级最高，& 次之，| 最低。

在遇到操作符时，根据优先级处理栈顶的操作符，确保当前操作符的优先级正确。

括号处理：

左括号 (直接入栈。

右括号) 触发括号内表达式的计算，直到遇到左括号 (。

栈的使用：

使用两个栈，一个存储布尔值，一个存储操作符。

在处理完一个表达式后，清空栈以便处理下一个表达式。

2.3 布尔表达式

2.3.1 问题描述

已知一个长度为 n ，仅含有字符 '(' 和 ')' 的字符串，请计算出最长的正确的括号子串的长度及起始位置，若存在多个，取第一个的起始位置。

子串是指任意长度的连续的字符序列。

例 1：对字符串 "((()))()" 来说，最长的子串是 "((()))"，所以长度=6，起始位置是 0。

例 2：对字符串 "()())" 来说，最长的子串是 "()", 子串长度=2，起始位置是 1。

例 3：对字符串 "" 来说，最长的子串是 ""，子串长度=0，空串的起始位置规定输出 0。

字符串长度： $0 \leq n \leq 1 \times 10^5$

对于 20% 的数据： $0 \leq n \leq 20$

对于 40% 的数据： $0 \leq n \leq 100$

对于 60% 的数据： $0 \leq n \leq 10000$

对于 100% 的数据： $0 \leq n \leq 100000$

下载并运行 p125_data.cpp 生成随机测试数据

提示：查找正确的括号子串可以用栈来实现，注意会有非法的右括号，比如例 2 中的第一个右括号。

2.3.2 基本要求

输入

一行字符串。

输出

子串长度，及起始位置

2.3.3 数据结构设计

使用了 `tuple` 库

```
tuple<int, int> LongestValidParentheses(const string& s) // 返回最长有效括号的长度和起始位置
{
    int maxLength = 0;
```



```

int startPosition = 0;
stack<int> stack;
stack.push(-1);

for (int i = 0; i < s.length(); ++i)
{
    if (s[i] == '(')
    {
        stack.push(i); //输入的是左括号，将其下标入栈
    }
    else
    {
        stack.pop(); //输入的是右括号，将栈顶元素出栈
        if (stack.empty())
        {
            stack.push(i); //栈为空，将当前右括号的下标入栈
        }
        else
        {
            int length = i - stack.top();
            if (length > maxLength)
            {
                maxLength = length;
                startPosition = stack.top() + 1;
            }
        }
    }
}

return make_tuple(maxLength, startPosition); //返回最长有效括号的长度和起始位置
}

```

2.3.4 功能说明（函数、类）

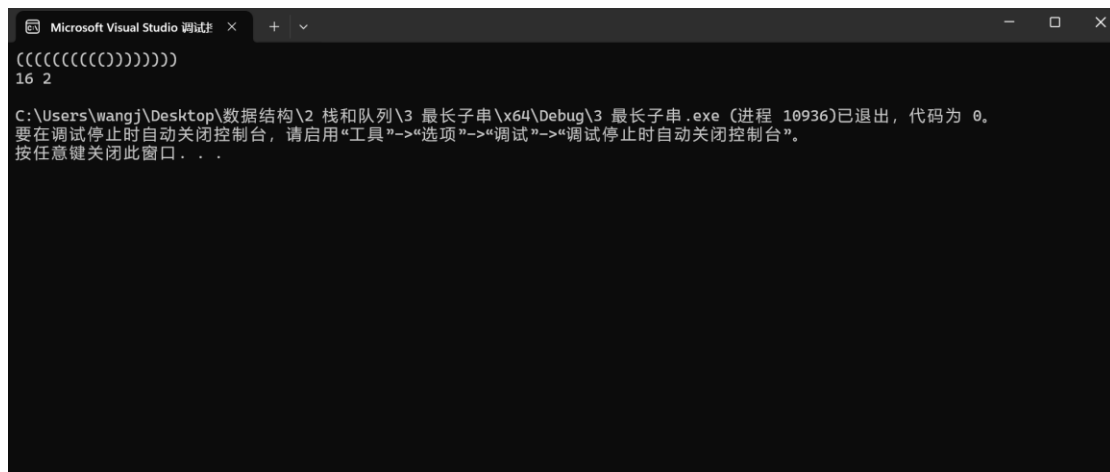
`tuple<int, int> LongestValidParentheses(const string& s)` //返回最长有效括号的长度和起始位置

```

int main()
{
    string input;
    cin >> input;
    auto result = LongestValidParentheses(input);
    cout << get<0>(result) << " " << get<1>(result) << endl;
    return 0;
}

```

2.3.5 调试分析（遇到的问题 and 解决方法）



2.3.6 总结和体会

这道题目要求计算一个仅包含字符 (和) 的字符串中最长的正确括号子串的长度及其起始位置。通过使用栈数据结构，我们可以有效地解决这个问题。

每次遇到右括号并且栈不为空时，计算当前有效括号子串的长度 $\text{length} = i - \text{stack.top}()$ 。如果当前长度大于 maxLength ，则更新 maxLength 和 startPosition 。最后返回最长有效括号子串的长度和起始位置。

2.4 队列的应用

2.4.1 问题描述

输入一个 $n \times m$ 的 0 1 矩阵，1 表示该位置有东西，0 表示该位置没有东西。所有四邻域联通的 1 算作一个区域，仅在矩阵边缘联通的不算作区域。求区域数。此算法在细胞计数上会经常用到。对于所有数据， $0 \leq n, m \leq 1000$

下载编译并运行 p44.cpp 生成随机测试数据

2.4.2 基本要求

输入

第 1 行 2 个正整数 n, m , 表示要输入的矩阵行数和列数

第 2—n+1 行为 $n \times m$ 的矩阵, 每个元素的值为 0 或 1。

输出

1 行, 代表区域数

2.4.3 数据结构设计

```
int n, m; //n行m列
int a[1000][1000]; //矩阵
bool visited[1000][1000]; //记录是否访问过
int area = 0; //记录连通块的个数

//四个方向
int dx[4] = { 0, 0, 1, -1 };
```

//四个方向

```
int dx[4] = { 0, 0, 1, -1 };
```

```
int dy[4] = { 1, -1, 0, 0 };
```

```
queue<pair<int, int>> q; //创建队列
q.push({ x, y }); //将起始点加入队列
visited[x][y] = true; //标记起始点为已访问
bool hasInternal = false; //标记是否有内部节点
```

2.4.4 功能说明（函数、类）

//检查坐标(x, y)是否有效且未访问过

```
bool isValid(int x, int y)
{
    return x >= 0 && x < n && y >= 0 && y < m && a[x][y] == 1
    && !visited[x][y];
}
```

//检查坐标(x, y)是否在矩阵边缘

```
bool isEdge(int x, int y)
{
    return x == 0 || x == n - 1 || y == 0 || y == m - 1;
}
```

//遍历连通块

```
bool bfs(int x, int y)
{
    queue<pair<int, int>> q; //创建队列
    q.push({ x, y }); //将起始点加入队列
    visited[x][y] = true; //标记起始点为已访问
    bool hasInternal = false; //标记是否有内部节点
    while (!q.empty())
    {
        int cx = q.front().first; //获取当前节点的x坐标
        int cy = q.front().second; //获取当前节点的y坐标
        q.pop(); //移除队列中的当前节点
        if (!isEdge(cx, cy))
        {
            hasInternal = true; //如果当前节点不在边缘，标记为有内部节点
        }
        //遍历四个方向的邻居节点
        for (int i = 0; i < 4; i++)
        {
            int nx = cx + dx[i]; //计算邻节点的x坐标
            int ny = cy + dy[i]; //计算邻节点的y坐标
            if (isValid(nx, ny))
            {
                visited[nx][ny] = true; //标记邻节点为已访问
                q.push({ nx, ny }); //将邻节点加入队列
            }
        }
    }
}
```

```

        }
    }
}
return hasInternal; //返回是否有内部节点
}
int main()
{
    cin >> n >> m; //输入矩阵的行数和列数
    for (int i = 0; i < n; i++)
    {
        for (int j = 0; j < m; j++)
        {
            cin >> a[i][j]; //输入矩阵元素
            visited[i][j] = false; //初始化访问标记为false
        }
    } //输入矩阵
    //遍历矩阵中的每个元素
    for (int i = 0; i < n; i++)
    {
        for (int j = 0; j < m; j++)
        {
            //如果当前元素为1且未访问过
            if (a[i][j] == 1 && !visited[i][j])
            {
                if (bfs(i, j))
                {
                    area++;
                } //如果有内部节点则计入区域数
            }
        }
    }
    cout << area << endl; //输出区域数
    return 0;
}

```

2.4.5 调试分析（遇到的问题 and 解决方法）



2.4.6 总结和体会

这道题想到了以前高程写过的一道题：我要吃饭！

广度优先搜索 (BFS)：题目中使用了 BFS 来遍历矩阵中的连通块。BFS 适合用于寻找最短路径或遍历所有节点，因为它逐层扩展，确保先访问距离起点最近的节点。

队列的使用：BFS 使用队列来存储待访问的节点。每次从队列中取出一个节点，访问其所有未访问的邻居节点，并将这些邻居节点加入队列。

2.5 队列中的最大值

2.5.1 问题描述

给定一个队列，有下列 3 个基本操作：

- (1) Enqueue(v): v 入队
- (2) Dequeue(): 使队首元素删除，并返回此元素
- (3) GetMax(): 返回队列中的最大元素

请设计一种数据结构和算法，让 GetMax 操作的时间复杂度尽可能地低。

要求运行时间不超过一秒

2.5.2 基本要求

输入

第 1 行 1 个正整数 n，表示队列的容量(队列中最多有 n 个元素)

接着读入多行，每一行执行一个动作。

若输入"dequeue"，表示出队，当队空时，输出一行"Queue is Empty";否则，输出出队的元素；

若输入"enqueue m"，表示将元素 m 入队,当队满时(入队前队列中元素已有 n 个)，输出"Queue is Full"，否则，不输出；

若输入"max",输出队列中最大元素，若队空，输出一行"Queue is Empty"。

若输入"quit",结束输入，输出队列中的所有元素

对于 20%的数据，有 $0 \leq n \leq 100$ ， $|\max(m) - \min(m)| \leq 1e4$ ；

对于 40%的数据，有 $0 \leq n \leq 6000$ ， $|\max(m) - \min(m)| \leq 1e6$ ；（未优化 $O(nm)$ 程序运行时间略微超过一秒）

对于 100%的数据，有 $0 \leq n \leq 10000$ ， $|\max(m) - \min(m)| \leq 1e8$ ；

对于每个测试点， $0x80000000 \leq \min(m) < \max(m) \leq 0x7fffffff$

对于每个测试点，操作的个数约为队列大小的 10 倍左右

事实上，测试数据保证对于后 80%左右的操作，队列内空位不会超过 15%

具体测试数据生成见 p101.py，下载并运行 p101.py 生成随机测试数据

输出

多行，分别是执行每次操作后的结果

2.5.3 数据结构设计

```
struct MaxQueue
{
    queue<int> q;          // 用于存储队列元素的普通队列
    deque<int> maxDeque;   // 用于存储当前最大值的两端队列
```

```
int capacity;           // 队列的容量
};
```

2.5.4 功能说明（函数、类）

```
// 构造函数，初始化队列容量
MaxQueue(int n) : capacity(n) {}

// 入队操作
void Enqueue(int v)
{
    if (q.size() >= capacity)
    {
        cout << "Queue is Full" << endl;
        return;
    }
    q.push(v);
    // 保持 maxDeque 中的元素从大到小排列
    while (!maxDeque.empty() && maxDeque.back() < v)
    {
        maxDeque.pop_back();
    }
    maxDeque.push_back(v);
}

// 出队操作
void Dequeue()
{
    if (q.empty())
    {
        cout << "Queue is Empty" << endl;
        return;
    }
    int removedValue = q.front();
    q.pop();
    cout << removedValue << endl;
    // 如果出队的元素是当前最大值，则从 maxDeque 中移除
    if (removedValue == maxDeque.front())
    {
        maxDeque.pop_front();
    }
}

// 获取队列中的最大值
void GetMax()
{
    if (q.empty())
```

```

    {
        cout << "Queue is Empty" << endl;
        return;
    }
    cout << maxDeque.front() << endl;
}
// 打印队列中的所有元素
void PrintQueue()
{
    queue<int> tempQueue = q; // 使用 q 以避免名称冲突
    while (!tempQueue.empty())
    {
        cout << tempQueue.front() << " ";
        tempQueue.pop();
    }
    cout << endl;
}

```

2.5.5 调试分析（遇到的问题解决方法）

```

4
dequeue
Queue is Empty
enqueue 10
enqueue 2
enqueue 3
max
10
dequeue
10
max
3
dequeue
2
enqueue 1
enqueue 2
enqueue 3
enqueue 4
Queue is Full
max
3
quit
3 1 2 3

C:\Users\wangj\Desktop\数据结构\2 栈和队列\5 队列中的最大值\x64\Debug\5 队列中的最大值.exe (进程 32844)已退出, 代码为 0
要在调试停止时自动关闭控制台, 请启用“工具”->“选项”->“调试”->“调试停止时自动关闭控制台”。
按任意键关闭此窗口...

```

2.5.6 总结和体会

使用 `queue<int>` 来存储队列的元素。

使用 `deque<int>` 来存储当前最大值，确保可以在常数时间内获取最大值。

从队列尾部取出元素，因此需要使用双端队列来实现。另外我们也需要一个辅助队列来记录所有被插入的值，以确定 `remove` 函数的返回值。保证了队列单调递减后，求最大值时只需要直接取双端队列中的第一项即可。

3. 实验总结

栈和队列都是基于线性存储结构的数据结构，但它们在数据操作顺序上有所不同。栈是后进先出的数据结构，主要用于实现程序的调用和返回等功能；而队列是先进先出的数据结构，主要用于实现进程调度和缓冲区管理等功能。